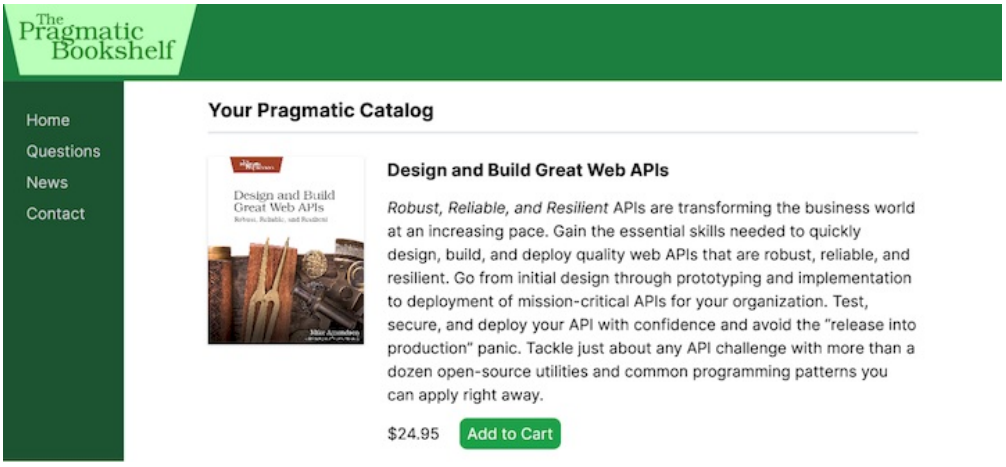
We also need to deal with two formatting issues. `button_to` creates an HTML `<form>` wrapping the `<button>`. HTML `<form>` is normally a block element that appears on the next line. We'd like to place them next to the price. This is no problem as Rails lets you specify both the `form_class` as well as the button `class`.

Now our index page looks like the following screenshot. But before we push the button, we need to modify the `create` method in the line items controller to expect a product ID as a form parameter. Here's where we start to see how important the `id` field is in our models. Rails identifies model objects (and the corresponding database rows) by their `id` fields. If we pass an ID to `create`, we're uniquely identifying the product to add.



Why the `create` method? The default HTTP method for a link is a `GET`, and for a button is a `POST`. Rails uses these conventions to determine which method to call. Refer to the comments inside the `app/controllers/line_items_controller.rb` file to see other conventions. We'll be making extensive use of these conventions inside the Depot application.

Now let's modify the `LineItemsController` to find the shopping cart for the current session (creating one if one isn't there already), add the selected product to that cart, and display the cart contents.

We use the `CurrentCart` concern we implemented in Iteration D1 to find (or create) a cart in the session:

rails7/depot_f/app/controllers/line_items_controller.rb